

Experiment 4: Implementation of Lamport Logical Clock in C

Aim

To implement **Lamport Logical Clock Algorithm** in C to simulate logical time ordering of events in a distributed system.

Theory

In a **Distributed System**, there is no global clock shared among processes. Each process has its own local clock, making it difficult to determine the correct order of events.

To solve this problem, **Leslie Lamport** introduced the **Lamport Logical Clock Algorithm** in 1978.

Key Concepts:

1. **Logical Clock:**
A counter maintained by each process to assign timestamps to events.
2. **Happened-Before Relation ($\rightarrow$):**
If event A occurs before event B, then $\text{timestamp}(A) < \text{timestamp}(B)$.
3. **Lamport Rules:**
 - **Rule 1 (Internal Event):**
Increment local clock by 1.
 - **Rule 2 (Send Event):**
Increment clock and send timestamp with message.
 - **Rule 3 (Receive Event):**
Update clock as:

$\text{Clock} = \max(\text{local_clock}, \text{received_timestamp}) + 1$

This ensures proper logical ordering of distributed events.

Algorithm

Step 1: Input

1. Read number of processes (p).
2. Read number of events per process (e).

Step 2: Initialization

3. Initialize all clock values to 0.

Step 3: Internal Events

4. For each process:
 - Assign first event timestamp = 1.
 - For subsequent events: $\text{clock}[i][j] = \text{clock}[i][j-1] + 1$

Step 4: Message Events

5. Read number of message events (m).
6. For each message:
 - Input sender process (sp) and event (se).
 - Input receiver process (rp) and event (re).
 - Update receiver clock:

$\text{clock}[\text{rp}][\text{re}] = \max(\text{clock}[\text{rp}][\text{re}], \text{clock}[\text{sp}][\text{se}] + 1)$

- Update all subsequent events in receiver process sequentially.

Step 5: Display Output

7. Print Lamport logical clock values for all processes.

Program

- `max()` function returns maximum of two values.
- 2D array `clock[p][e]` stores timestamps.
- First assigns sequential timestamps.
- Then adjusts timestamps for message events using Lamport rules.
- Finally displays updated logical clocks.

Output

If:

- Processes = 2
- Events per process = 3

- Message: P0E1 $\rightarrow$ P1E1

The updated timestamps will reflect causal ordering.

Conclusion

The experiment successfully demonstrates the working of Lamport Logical Clock Algorithm.

It shows how Logical timestamps maintain event ordering and Message passing affects time synchronization